

MVC (Model-View-Controller) Design Pattern

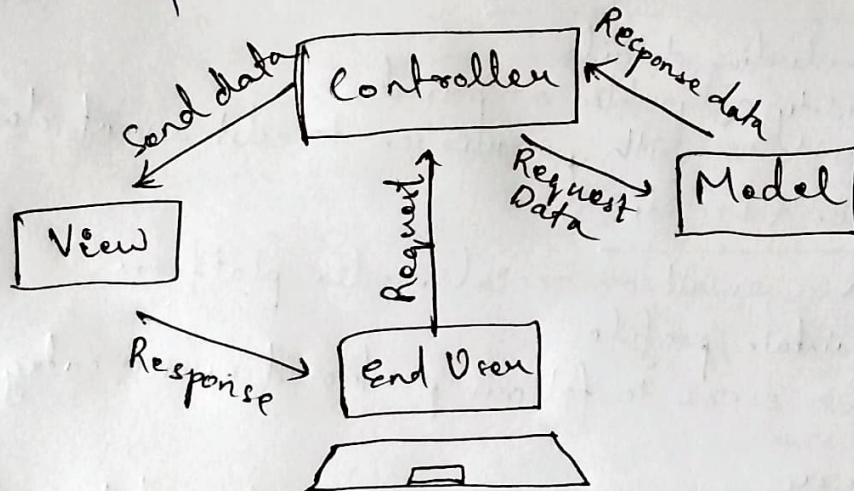
MVC is an architectural style that divides an application into three key components that work together:

Model: Responsible for managing data and the underlying business logic.

View: Takes care of the user interface and presentation aspects.

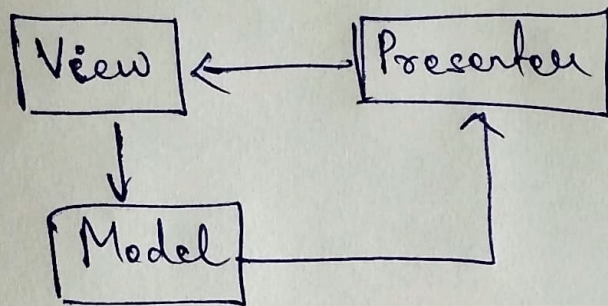
Controller: Handles user interactions and acts as a bridge between the model and the View.

This structure simplifies complex applications, encourages clean code organisation and enables multiple visual representation of the same data.



MVC Variants

1. MVP (Model View Presenter)



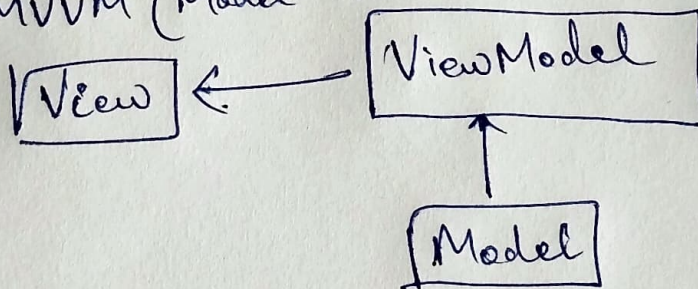
Key differences from MVC

View is more passive and does not observe the Model directly.
Presenter handles View explicitly.
View & Model are completely decoupled.

Use case for MVP

If you want an application requiring extensive unit testing, need clear separation between UI & business logic and in UI frameworks where testing the View is challenging.

2. MVVM (Model-View-View-Model)



Key differences: Introduces data binding between View and ViewModel.
ViewModel exposes properties and commands that View can bind to.
View has no knowledge of the Model.

When to Use MVVM: In UI frameworks that support data-binding when we want to leverage reactive programming for complex UIs that would benefit from state management.